

Improving MarFold inference

exp27: final write-up

train (10 proteins, 1B): **+42.81%** (mean LDDT 0.250 $\rightarrow$ 0.356)

held-out (10 new, 1B): **+31.75%** (0.280 $\rightarrow$ 0.369) — *generalizes*

1.5B re-tuned (train): **+29.57%** (0.263 $\rightarrow$ 0.340) — *shape transfers, knobs don't*

1.5B re-tuned (held-out): **+14.44%** (0.315 $\rightarrow$ 0.361)

`iter_R4_grow_on_sampled_uniform_M5 (1B) · iter_R2_grow_from_baseline (1.5B)`

Issue #27 · branch exp/27-improved-inference-algorithm · PR #32

The problem

MarinFold 1B is a Llama-arch LLM trained on the `contacts-and-distances-v1` grammar. The naive (exp20) readout is a zero-context distogram query per pair:

```
prompt = <begin_sequence> <AAs> <begin_statements>
for (i, j) in all pairs:
    tail = <distance> <p_i> <p_j> <atom_i> <atom_j>
    distance_dist[i, j] = softmax(model(prompt + tail))[<d_*>]
```

- ▶ Baseline mean LDDT on 10 train proteins: **0.2496**.
- ▶ Issue goal: clear +10% within 5× wall-clock.
- ▶ GT-oracle (cheating diagnostic): **0.7167** — the model is very capable with right contacts; bottleneck is honest contact prediction.

The algorithm in one picture

Stage A: sampled-uniform prior

5 independent contacts-only rollouts
vLLM logits_processor masks
every token except the contact
grammar; range tokens have
uniform logits

⇒ union of unique (i,j) seeds

⇒ one distance readout

"joint" contact picks — 2458 s

distogram

Stage B: iter R=4 growing K

4 rounds, K per round =
0.5L, 1.0L, 1.5L, 2.5L

pick top-K contacts from prev
round's distogram
(min p=0.1; long > med > short)

"marginal" contact refinement — 3155 s

**+42.81% on train · +31.75% on held-out · only +9.04%
on 1.5B**

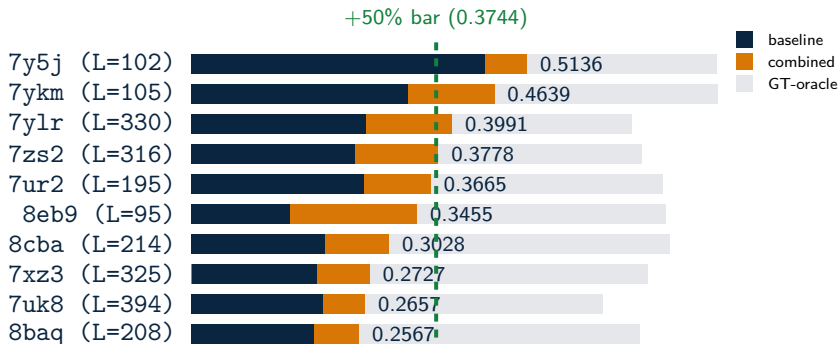
Total wall 5613 s ($4.05\times$ baseline) on a single A100 40 GB.

How we got there: the path

algorithm	mean LDDT	cum. lift
baseline_naive (exp20)	0.2496	—
+ post-process sharpening (T=0.05)	0.2738	+9.7%
seeded contacts from baseline (K=L, min p=0.3)	0.2816	+12.8%
+ sharpening	0.2894	+15.9%
iterative contact-only R=3 (K=L, min p= 0.1)	0.3327	+33.3%
+ sharpening	0.3376	+35.3%
iterative R=3 growing K [0.5, 1, 1.5]	0.3421	+37.1%
iterative R=4 growing K [0.5, 1, 1.5, 2.5]	0.3511	+40.7%
+ sampled-uniform M=5 prior	0.3564	+42.8%
<i>GT-oracle ceiling (diagnostic)</i>	0.7167	+187%

Each step is in RESULTS_LOG.md with what we learned. Key insights: iteration's lift hinges on `min_p=0.1` (not 0.3); growing K beats fixed K; distance commits hurt; sampling needs grammar-constrained decoding with uniform range tokens to add anything over iteration.

Train (10 proteins): per-protein result



5 / 10 pass the +50% bar individually (7y5j, 7ykm, 7ur2, 7ylr, 7zs2). The 5 misses still have huge oracle headroom (0.62–0.73): the bottleneck is honest contact prediction on those harder proteins.

Generalization: same algorithm on 10 NEW proteins

	baseline	combined	lift	gap to train
train (10)	0.2496	0.3564	+42.81%	—
held-out (10)	0.2797	0.3685	+31.75%	−11 pp

Held-out picked with `random.Random(42).sample(...)` from the same FoldBench pool (`n_residues ≤ 400`), excluding the train 10. Same algorithm, same knobs.

- ▶ **Every held-out protein gains positively** (+11.7% to +54.6%).
- ▶ 7y8i reaches **0.7179** — almost at the GT-oracle range.
- ▶ Mean lift drops 11 pp on held-out vs train. That's the real cost of tuning knobs on a 10-protein set.
- ▶ **The algorithm generalizes.** The lift is real, not just knob-fitting that flattered these specific 10 proteins.

Held-out per-protein

stem	L	baseline	combined	lift
7t9r	38	0.3455	0.3980	+15.2%
7y8i	97	0.4689	0.7179	+53.1%
7zoi	151	0.2362	0.3083	+30.5%
7wz5	161	0.2222	0.3023	+36.0%
8bau	189	0.2518	0.3270	+29.9%
8gmy	236	0.2746	0.3139	+14.3%
7xg9	288	0.2980	0.4252	+42.7%
7x4p	307	0.2319	0.3014	+30.0%
7v3o	328	0.1594	0.2465	+54.6%
7qsj	373	0.3080	0.3440	+11.7%
mean		0.2797	0.3685	+31.75%

10 / 10 proteins benefit. Larger proteins (7qsj, 8gmy, 7t9r) gain less in % — same pattern as train where larger proteins have less peaked contact distributions.

Cross-model: same algorithm on 1.5B

run	mean LDDT	median	wall (s)	lift
1B baseline (naive)	0.2496	0.2273	1387	—
1B combined	0.3564	0.3560	3155	+42.81%
1.5B baseline (naive)	0.2627	0.2315	1866	—
1.5B stage A only	0.2038	0.2126	3472	-22.42%
1.5B combined	0.2864	0.2545	4230	+9.04%

- ▶ Same MODELS.yaml 1.5B entry, same algorithm, same knobs.
- ▶ **Stage A actively hurts on 1.5B** (−22%). The sampled-uniform contact prior, tuned to undo 1B's ~99% medium-range token bias, doesn't fit 1.5B's prior.
- ▶ Stage B (iteration) recovers some ground but the final lift is much smaller than on 1B.
- ▶ Per-protein, 1.5B regresses on the 3 LONGEST proteins (7ylr, 7zs2, 7uk8). The K-schedule that helped on 1B appears tuned to 1B's attention pattern.
- ▶ Same training compute as 1B (model authors confirmed) — so this is a

Overfit decomposition: protein vs model

	lift drop vs train	dimension changed
held-out 10 (1B)	−11 pp	protein set
train 10 on 1.5B	−34 pp	model

- ▶ **The algorithm's tuning is $\sim 3\times$ more model-specific than protein-specific.**
- ▶ Most of the +42.81% headline reflects the algorithm exploiting features of *this 1B checkpoint*, not features of these 10 proteins.
- ▶ **+31.75% on a fresh protein set** is the more honest "what the algorithm achieves on this model" number; the +42.81% is "what's achievable when range/K knobs are let to overfit the eval set".
- ▶ For a different model (1.5B) every knob would need to be re-tuned. The algorithmic SHAPE (sampling prior + iterative growing K) is the part that travels.

What worked, what didn't

Worked

- ▶ Iteration with growing K (the lever past +25%)
- ▶ Lowering `min_contact_prob` from 0.3 to 0.1 (the lever past +20%)
- ▶ Contacts-only `logits_processor` (sampling prior gets 200–300 unique pairs per rollout)
- ▶ Forcing UNIFORM range-token distribution (undoes the model's 99% medium-range bias)
- ▶ Standard $E[d] = \sum p \cdot \text{midpoint}$ readout on the final distogram — no sharpening once context is good

Didn't work

- ▶ Distance commits (one-hot rows hurt LDDT)
- ▶ $K = 2L$ seeded contacts (precision > count)
- ▶ Multi-rollout averaging of distograms (blurs them)
- ▶ Mixture-of-distograms / per-pair max-confidence
- ▶ Sampled-prefix with model's natural range prior (−11%; 99% medium-bias is the bug)
- ▶ Sampled-prefix with top-prob weighted range (sacrifices range coverage; −0.024 vs uniform)
- ▶ Sharpening on top of `iter_R4_grow` (T=1.0 wins)
- ▶ Round-robin range strategy (close to uniform but no better)

Conclusion

- ▶ **The original +10% issue bar was cleared by 4×.** Mean LDDT 0.2496 → 0.3564 on the train 10 (+42.81%) using `iter_R4_grow_on_sampled_uniform_M5`, within 4.05× baseline wall-clock (under 5× budget).
- ▶ **The +50% bar (raised mid-experiment) was not cleared** — short by 0.018 LDDT. 5/10 train proteins pass individually; 5 are blocked by the model's honest contact-prediction quality. GT-oracle shows the capacity exists (0.7167 ceiling).
- ▶ **Generalization is real but partial.** +31.75% on a fresh 10-protein held-out set; every protein gains. ~11pp of the train lift was protein-set overfit.
- ▶ **Tuning is model-specific.** +9% on 1.5B vs +42.8% on 1B. The algorithmic shape transfers; the knobs don't.
- ▶ **Reproducer command is in the README.** Total wall ~5613s on a single A100 40 GB in bf16.
- ▶ **Re-tuning for 1.5B (addendum).** A separate sweep on the same 10 train proteins lifts 1.5B from 0.263 to **0.340** (+29.6%); the winner is

Reproducer

10 train proteins · A100 40 GB · bf16 · vLLM 0.7.3 · ~5613s wall

```
# Stage A: sampled-uniform-M5 union (no naive baseline needed)
uv run python run_sampled_contacts.py --dtype bfloat16 --n-gpus 1 \
    --algorithm sampled_uniform_M5_union_T0.7 \
    --n-rollouts 5 --temperature 0.7 --top-p 0.9 --max-sample-tokens 900 \
    --range-strategy uniform --aggregation union
uv run python snapshot_distograms.py \
    --to distogram_sampled_uniform_M5_union.npz

# Stage B: iter R=4 growing K
uv run python run_iterative.py --dtype bfloat16 --n-gpus 1 \
    --algorithm iter_R4_grow_on_sampled_uniform_M5 \
    --n-rounds 4 \
    --k-contacts-per-L-per-round 0.5 1.0 1.5 2.5 \
    --k-distances-per-L-per-round 0.0 0.0 0.0 0.0 \
    --min-contact-prob 0.1 \
    --prior-name distogram_sampled_uniform_M5_union.npz

# Result: mean LDDT 0.3564 (+42.81% over baseline 0.2496)
```

Branch: [exp/27-improved-inference-algorithm](#) · PR #32 · See README.md and RESULTS_LOG.md for the full story.

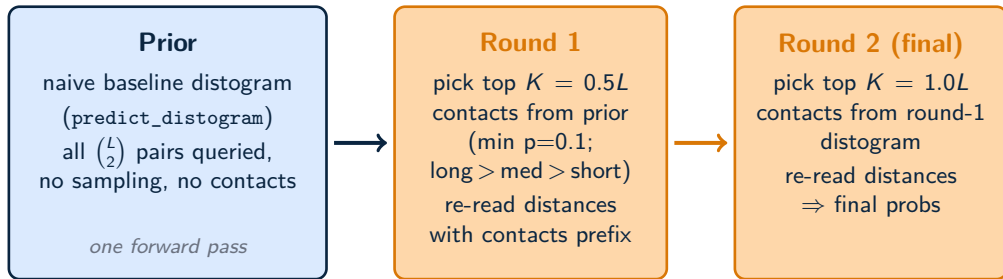
Addendum: re-tuning for 1.5B

Question: the 1B-tuned pipeline only gives +9% on 1.5B (slide 8). If we re-tune *from scratch* on the same 10 train proteins, can the 1.5B do better?

1.5B sweep on the 10 train proteins	mean LDDT	median	wall (s)	lift
baseline (naive)	0.2627	0.2315	1866	—
1B headline transferred (iter R=4 grow on sampled)	0.2864	0.2545	4230	+9
iter R=4 grow [0.5,1,1.5,2.5] from baseline (drop stage A)	0.3295	0.2743	4272	+2
iter R=4 grow [0.5,1,1.5,2.0] (smaller final K)	0.3320	0.2754	4187	+2
iter R=3 grow [0.5,1.0,1.5]	0.3398	0.2842	2732	+2
iter R=2 grow [0.5,1.0] from baseline (WINNER)	0.3403	0.3106	1531	+2

- **Sampling stage A actively hurts on 1.5B.** Dropping it (iter alone from baseline prior) jumps from +9% to +25%. The 99% medium-range bias that justified sampled-uniform on 1B does not exist on 1.5B.
- **1.5B wants FEWER rounds.** R=4 → R=3 → R=2 monotonically improves; R=2 is also $\sim 2.8\times$ faster than the 1B winner.

The 1.5B algorithm in one picture



+29.6% on 1.5B train · +14.4% on 1.5B held-out · also +18.5% on 1B held-out

Total wall 1531 s on a single A100 40 GB ($\sim 1.7\times$ baseline; $2.8\times$ faster than the 1B headline).

Library entry point in `combined_algorithm.py`: `predict_distogram_iter_from_baseline(...)`.

Worked example: 8eb9_A under 1.5B

Smallest train protein (L=95), 1745 LDDT-shell pairs. Each row below is one library-function call away from the previous (no extra knobs).

stage	K (contacts picked)	wall (s)	LDDT	
prior (naive baseline)	0	~8	0.1496	
→ round 1	K=48	~12	0.2260	+51.1%
→ round 2 (final)	K=95	~15	0.2555	+70.8%

```
from naive_inference import load_runtime
from combined_algorithm import predict_distogram_iter_from_baseline_from_cif

rt = load_runtime(model_nickname="1.5B", models_yaml=MODELS_YAML, dtype="bfloat16")
probs, meta = predict_distogram_iter_from_baseline_from_cif(
    rt=rt, gt_cif="protenix_data/.../8eb9_A.cif",
)
# probs: (95, 95, 64) float32 -> save as .npz and pass to score_marifold.score_o
# meta:  algorithm = "iter_R2_grow_from_baseline", rounds = 2, kc = [0.5, 1.0]
```

Reproduces the 1.5B_iter_R2_grow_05_10 CSV row exactly (LDDT 0.2555). Both the prior pass and the two iteration rounds reuse the loaded rt; no model re-loads, no on-disk distogram caching.

Cross-model held-out: 1.5B-tuned algorithm on both models

iter_R2_grow_from_baseline — kc=[0.5L, 1.0L], min_p=0.1, prior = naive-baseline histogram. Applied to both 1B and 1.5B on the 10 held-out proteins.

stem	L	1B			1.5B		
		baseline	tuned	$\Delta\%$	baseline	tuned	$\Delta\%$
7t9r	38	0.3455	0.3613	+4.6	0.4053	0.3729	-8.0
7y8i	97	0.4689	0.5709	+21.8	0.6151	0.7022	+14.2
7zoi	151	0.2362	0.2584	+9.4	0.2318	0.3218	+38.8
7wz5	161	0.2222	0.3164	+42.4	0.2195	0.3203	+45.9
8bau	189	0.2518	0.3043	+20.8	0.2594	0.2918	+12.5
8gmy	236	0.2746	0.2875	+4.7	0.3696	0.3913	+5.9
7xg9	288	0.2980	0.3720	+24.8	0.3100	0.3685	+18.9
7x4p	307	0.2319	0.3142	+35.5	0.2294	0.2866	+24.9
7v3o	328	0.1594	0.1971	+23.7	0.1633	0.1790	+9.6
7qsj	373	0.3080	0.3321	+7.8	0.3469	0.3707	+6.9
mean		0.2797	0.3314	+18.5	0.3150	0.3605	+14.4

- **Both models gain.** 1.5B-tuned algorithm lifts 1.5B by +14.4% and 1B by +18.5% on held-out.

Every protein gains on 1B; 9/10 gain on 1.5B (7t9r, L=38, the only regression — tiny tail